Assignment 4: Lenia MPI report

Nejc Krajšek, Rok Mušič

May 2026

Implementation. We implemented the basic MPI version with a row-wise split of the world. Rank 0 initializes the full world and distributes contiguous row blocks with `MPI_Scatterv`. Each process computes only its own rows. Since the kernel size is 26×26, each process needs a halo of 13 rows above and below its local block. We exchange those rows with `MPI_Sendrecv` in a ring layout. One important detail is that a single neighbor exchange is not enough for the smallest case with many processes: for example, at 128×128 and 32 processes, each process owns only 4 rows, so the halo has to be collected over several rounds. After the last step, the final world is gathered with `MPI_Gatherv`. The optional GIF mode also uses gather to rank 0, but we did not include it in the timings because it would add extra communication and file output every step.

Runtime results. Table 1 shows the measured execution times. The general picture is what we expect. For 2 and 4 processes, the results are almost ideal for all larger sizes: the time is very close to half and quarter of the single-process time. The smallest case also scales well at first, but after 4 processes the benefit becomes much smaller, which is also expected because communication becomes a larger part of the total runtime when each process has very little work.

Table 1: Mean runtime ± sample standard deviation [s] over 5 runs.

Size	$p = 1$	$p = 2$	$p = 4$	$p = 16$	$p = 32$
128×128	1.100 ± 0.001	0.556 ± 0.001	0.286 ± 0.003	0.107 ± 0.023	0.082 ± 0.002
512×512	17.554 ± 0.012	8.792 ± 0.006	5.266 ± 1.921	1.375 ± 0.444	0.867 ± 0.259
1024×1024	70.275 ± 0.048	35.189 ± 0.025	17.582 ± 0.032	4.507 ± 0.026	3.360 ± 1.105
2048×2048	293.215 ± 0.504	146.445 ± 0.178	70.750 ± 0.401	18.884 ± 0.679	10.289 ± 0.168
4096×4096	1548.189 ± 12.932	763.408 ± 1.785	376.515 ± 1.253	92.017 ± 1.669	47.705 ± 0.321

There are a few points with visibly larger deviation: 128/16, 512/4, 512/16, 512/32, and 1024/32. In those cases one or two runs were noticeably slower than the others. This does not look like a real change in the algorithm, because the surrounding points are stable and the larger cases are very consistent. A more likely explanation is normal cluster noise, process placement, or temporary contention on the node. For that reason, we kept the required average over several runs and report the standard deviation next to it.

Speed-up. Table 2 shows the speed-up computed from the average times. The results again make sense. For 2 processes the speed-up is essentially ideal in all cases, and for 4 processes it is also very close to ideal. The difference appears at 16 and 32 processes. At 128×128, the speed-up reaches only $10.287\times$ and $13.432\times$, which is far from ideal but fully expected: the local blocks are very small, communication takes several halo rounds, and the computation per process is too small to hide that overhead.

At larger sizes the picture improves steadily. For 2048×2048, the code reaches $15.527\times$ on 16 processes and $28.499\times$ on 32 processes. For 4096×4096, it reaches $16.825\times$ on 16 processes and $32.454\times$ on 32 processes. The last value is slightly above perfect linear speed-up. We do not interpret that as a special effect of the algorithm; it is more likely a small measurement artifact, helped by cache effects and normal run-to-run variation.

Table 2: Speed-up computed from the average times.

Size	$p = 1$	$p = 2$	$p = 4$	$p = 16$	$p = 32$
128×128	1.000	1.977	3.850	10.287	13.432
512×512	1.000	1.997	3.333	12.764	20.246
1024×1024	1.000	1.997	3.997	15.591	20.914
2048×2048	1.000	2.002	4.144	15.527	28.499
4096×4096	1.000	2.028	4.112	16.825	32.454

Overall, the results match expectation well. The code scales very nicely for medium and large grids, while the smallest grid loses efficiency at high process counts because the communication cost becomes too large relative to the work. The few noisier points are easy to spot from the standard deviations and do not change the main conclusion: the MPI version is effective, and its behavior is in line with what we would expect from a row-wise stencil computation with halo exchange.